lsqvce 6.16.1 — API Reference

least-squares adjustment with variance component estimation

Contents

1 The `data` struct **2**
- 1.1 The streaming kind . . . 2
- 1.2 The matrices kind . . . 4
- 1.3 The batched kind . . . 5

2 `buildDay` — the function you write **5**
- 2.1 Signature . . . 5
- 2.2 The `info` argument . . . 6
- 2.3 The return value: one cell per arc . . . 6
- 2.4 The arc schedule — `systems(k).arcs` . . . 7
- 2.5 Checks while reading . . . 7

3 Covariance specifications **8**
- 3.1 Laws . . . 8
- 3.2 Policy fields . . . 9
- 3.3 How a law is whitened . . . 9

4 `lsqvce.Options` **9**
- 4.1 What to estimate . . . 10
- 4.2 The loop . . . 11
- 4.3 Run control . . . 12
- 4.4 The groups . . . 12

5 Priors **14**
- 5.1 Fields . . . 14
- 5.2 Rules . . . 15
- 5.3 Where the result is reported . . . 15
- 5.4 From a parameter table . . . 16

6 `lsqvce.Results` **16**
- 6.1 Properties . . . 16
- 6.2 Methods . . . 17

7 Help of the user-tier units **17**
- 7.1 lsqvce.fit . . . 17
- 7.2 lsqvce.Options . . . 20
- 7.3 lsqvce.Results . . . 22
- 7.4 lsqvce.report . . . 24
- 7.5 lsqvce.layout . . . 25

8 The engine, kernel and util tiers **26**
- 8.1 lsqvce.engine — drivers, accumulator, loaders, sweeps, diagnostics . . . 26

8.2 lsqvce.kernel — the numerical kernels 27
8.3 lsqvce.util — plumbing 28

9 Error and warning identifiers **28**

1 The data struct

Everything the toolbox knows about a problem enters through one plain MATLAB struct, `data`, handed to `lsqvce.fit(data, opts)`. `fit` tells three **kinds** apart by the fields present:

kind	fields	for
streaming	`commonParams`, `days`, `systems` (+ `priors`, `paramColumns`)	arc-wise data read day by day through a `buildDay` function, local unknowns eliminated on the fly, estimated stochastic models
matrices	`A`, `l`, `p` (+ `priors`)	a design matrix that fits in memory, no local unknowns, diagonal weights
batched	`batchFcn`, `numBatches`, `numUnknowns` or `x0` (+ `priors`)	a design too large for one matrix, or a nonlinear model

`data` is a **scalar struct**; field names are case-sensitive (lowerCamelCase). An unknown field is an error (`lsqvce:fit:unknownField`), so a misspelt field cannot be ignored silently.

1.1 The streaming kind

field	type / size	required	meaning
`commonParams`	cell R×C, C ≥ 1	yes	the common parameter **templates** (below)
`days`	double 1×D	unless every system has its own `days`	the default calendar: integer day identifiers (the loader cache is keyed on them)
`systems`	struct 1×M, M ≥ 1	yes	one entry per observing system (below)
`priors`	struct array	no	prior groups on the common parameters (chapter Priors)
`paramColumns`	struct	no	which table columns hold a row's validity and its name (below)

1.1.1 `commonParams` — the common parameter table

One row per **template**. **Column 1 is the scope token**, columns 2…C are your own convention (degree, order, name, epoch …) and are passed through to `buildDay`:

token	instantiated as	typical use
`'common'`	one column of A, always	spherical-harmonic coefficients
`'day'`	one column per calendar day that carries at least one arc in at least one system	a daily common parameter

`'arc'` is not allowed in this table (`lsqvce:fit:badScope`): arcs belong to a system. nc, the number of columns of A, is **derived** from the expansion and never declared.

Column layout of A. The `'common'` rows first, in table order; then one day block per calendar day, in calendar order (the sorted union of every system's day list). `buildDay` receives the position of the current day's block as `info.commonDayOffset` / `info.commonDayCount`. A minimal table: `[repmat({'common'}, nc, 1), num2cell((1:nc).')]`.

1.1.2 `systems(k)` — one entry per observing system

All entries share the same fields; fill an unused optional field with `[]`.

field	type / size	default	meaning
`buildDay`	function handle	*(required)*	your day builder (chapter `buildDay`)
`covariance`	struct	*(required)*	one field per observation type holding a covariance spec (chapter Covariance specifications). **The fieldnames are the observation types**, in processing order; `buildDay` is called once per (day, type).
`localParams`	cell R×C	`{}`	the local parameter templates (below); not declared → nl = 0
`arcs`	double, struct or function handle	*(none)*	the arc schedule (chapter `buildDay`); required only by `'arc'` template rows
`days`	double 1×D	`data.days`	this system's calendar
`id`	double scalar or single char	the index k	identifier, unique across systems
`name`	char / string	`''`	label used in reports
`processingOptions`	struct	`struct()`	anything `buildDay` needs (paths, force models); its 5th argument
`priors`	struct array	none	prior groups on this system's local parameters (chapter Priors)

`buildDay` + `covariance` is a complete system.

1.1.3 `localParams` — the local parameter table

Column 1 is the scope token, columns 2…C your convention:

token	one block per …	typical use
`'arc'`	(day, arc) of the system	initial state, per-arc empirical parameters
`'day'`	day of the system that carries ≥ 1 arc — every calendar day when no schedule is declared	accelerometer scale / bias / drift
`'system'`	whole adjustment	instrument constants

`'arc'` rows require a declared arc schedule; `'day'` rows without a schedule live on every calendar day of the system; `'system'`-only tables need neither. nl is derived.

Column layout of B (per system). Per day, in calendar order: the arc blocks in sorted arc-id order (each block = the `'arc'` rows in table order), then the `'day'` block; after all days one trailing `'system'` block. A day whose declared schedule names no arc gets no block at all. `buildDay` never computes an offset: every offset arrives in its `info` argument.

A template has the same rows for every arc. Arcs with genuinely different parameter counts are an engine-tier feature (`lsqvce.engine.makeLoader` with `LocalsPerArc`).

1.1.4 `paramColumns` — validity and names

field	meaning
`start`, `end`	the columns of both tables that hold a row's validity (`[MJD seconds]` pairs or fractional MJD; `[]` = always valid). A `'day'` / `'arc'` row is then instantiated only on calendar days its validity overlaps (start < d+1 and end > d); `'common'` / `'system'` rows are always present.
`name`	the column holding a row's name, matched by the `group` field of a prior (default 2)

Without `paramColumns` every row is valid always. `buildDay`'s `info` then lists which template rows exist on the day (`info.arcRows`, `info.dayRows`, `info.sysRows`, `info.commonDayRows`).

1.1.5 `lsqvce.layout(data)` — the columns announced

```
lay = lsqvce.layout(data);
lay.nc, lay.common(j)                    % template row (and day) of column j of A
lay.systems(k).nl, lay.systems(k).local(c)    % (row, day, arc) of column c of B
c = lay.systems(k).col(row, day, arc);         % lookup, 0 = not instantiated
c = lay.colCommon(row, day);
```

This is the column map a prior operator `L` is written in, and the one `R.toTable` uses to write estimates back into your tables.

1.2 The matrices kind

field	size	meaning
`A`	m×u	design matrix
`l`	m×1	observed minus computed
`p`	m×1	weights (the diagonal of P); absent = unit weights
`priors`	struct array	prior groups (chapter Priors)

1.3 The batched kind

field	size	meaning
`batchFcn`	function handle	`[A, l, p] = batchFcn(k)`, k = 1…`numBatches`: one batch of design rows, observations and weights. With `x0`: `[A, w, p] = batchFcn(k, x)` returns the Jacobian at `x` and `w = l − f(x)`.
`numBatches`	scalar	the number of batches
`numUnknowns`	scalar	u, linear form only
`x0`	u×1	the starting point; makes the problem **nonlinear** (damped Gauss–Newton). u = `numel(x0)`; `numUnknowns` must then be absent (`lsqvce:fit:uCount`).
`priors`	struct array	prior groups (chapter Priors)

Both in-memory kinds run with `Estimate = "none"` (variance components need the streams of a streaming system, `lsqvce:fit:modelKind`); `Robust` selects M-estimation. Their per-observation diagnostics are in the whitened frame, `R.diag.v = sqrt(p).*v`.

2 `buildDay` — the function you write

2.1 Signature

`fit` passes as many of these arguments as the handle declares (`nargin`); a `varargin` builder receives all six:

```
D = buildDay(day, obsType)
D = buildDay(day, obsType, common, local, procOpt)
D = buildDay(day, obsType, common, local, procOpt, info)
```

argument	what it is
`day`	one entry of the system's calendar (numeric)
`obsType`	one entry of `fieldnames(covariance)` (char)
`common`	`data.commonParams`
`local`	this system's `localParams` (or `{}`)
`procOpt`	this system's `processingOptions`
`info`	this day's expansion: every column offset of A and B (below); `[]` when nothing is scoped

`fit` calls `buildDay` once per (day, observation type). Every pass over the data reads each day twice (accumulation, then the residual sweep), and an iterative run multiplies that by its passes. Hence two rules:

1. **Determinism.** The same arguments must give byte-identical output: no random numbers, clocks or mutable globals.
2. **Cache expensive reads yourself.** The toolbox keeps `Options.CacheDays` built days resident (default one); a raw file read shared by several observation types belongs in your own cache, in the closure that binds `buildDay`.

2.2 The `info` argument

```
info.arcId            % 1xA  ids of this day's arcs
info.offsetOf         % @(arcId) -> column offset of that arc's block in B
info.count            % 1xA  columns of each arc block (= number of 'arc' rows)
info.dayOffset        % 1x1  this day's 'day' block offset in B ([] if none)
info.dayCount         % 1x1  its width (0 if none)
info.sysOffset        % 1x1  the 'system' block offset in B
info.sysCount         % 1x1  its width
info.nl               % 1x1  total local count of the system
info.nc               % 1x1  total common count
info.commonCount      % 1x1  width of A's leading 'common' block
info.commonDayOffset  % 1x1  this day's common day-block offset in A ([] if none)
info.commonDayCount   % 1x1  its width (0 if none)
info.arcRows, info.dayRows, info.sysRows, info.commonDayRows
                      %      template rows that exist on this day (with paramColumns)
```

Fill `B(rows, info.offsetOf(id) + (1:k))` and `A(:, info.commonDayOffset + (1:w))`.

2.3 The return value: one cell per arc

Every field is a **1 × nArc cell**; cell a is arc a. A day with one arc returns `{A}`; a day without data returns empty cells.

field	each cell	meaning
`A`	m_a × nc	design matrix of the common unknowns
`B`	m_a × nl	design matrix of the local unknowns; may be `sparse`
`l`	m_a × 1	observed minus computed (the reduced observation)
`t`	m_a × 1	epoch of each row in **seconds** (the whitener needs the lags)
`Cep`	n_a×1, n_a×d or n_a×d×d	*optional* — the per-epoch accuracy of the arc (below)
`nAxis`	scalar	*optional* — the axis count of a multi-axis stream without `Cep`

Arc lengths may differ freely. An arc whose cells have 0 rows is a **data gap** and is skipped. Return **raw, unwhitened** matrices: the toolbox whitens every arc from the covariance spec and reweights it during the variance-component loop. No other field is allowed (`lsqvce:makeLoader:badDay`).

2.3.1 `Cep` — the per-epoch accuracy

shape	meaning	the numbers are
n_a × 1	scalar stream with per-epoch accuracy	standard deviations
n_a × d	multi-axis stream, axes uncorrelated within an epoch	standard deviations per axis
n_a × d × d	multi-axis stream, full per-epoch covariance	covariance matrices

The axis count d is `size(Cep, 2)`, the same for the whole day; n_a·d must equal the arc's row count. Rows of a multi-axis stream are **epoch-major** (x_1 y_1 z_1 x_2 ...) and every arc holds whole epochs. `Cep` is relative to the level `sigma` of the covariance spec: declare `sigma = 1` when `Cep` already carries the absolute accuracy. A multi-axis day without `Cep` is whitened with D = I and reported once per run (`lsqvce:makeLoader:noCep`); a `var` law takes no `Cep`.

2.4 The arc schedule — `systems(k).arcs`

form	meaning
omitted	the arcs of a day are the cells `buildDay` returns (ids 1…nArc); days may differ in their count. `'day'` rows live on every calendar day; a calendar day returning no arc in any stream is refused (`lsqvce:makeLoader:emptyDay`).
`4` or `[1 2 4]`	static: these arc ids on every day
`struct('kbr', 2, 'pos', 4)`	static, per observation type
`@(day, obsType) ids`	a resolver returning the ids of a day *without reading data* (ragged schedules, gaps, manoeuvres)

With a declared schedule cell a is the a-th scheduled id, and the number of cells must match (`lsqvce:makeLoader:arcCount`). An arc a scheduled day does not carry is returned as an empty cell. `'arc'` template rows require a declared schedule.

2.5 Checks while reading

identifier	meaning
`lsqvce:makeLoader:buildDay`	`buildDay` threw; its error is attached as the cause
`lsqvce:makeLoader:badDay`	a required field is missing or not a 1×nArc cell, or an unknown field is present
`lsqvce:makeLoader:dayShape`	the cells disagree in number, or an arc's `A`, `B`, `l`, `t` disagree in rows
`lsqvce:accum:arcBadNc` / `arcBadNl`	`A` or `B` has the wrong number of columns
`lsqvce:makeLoader:arcCount`	a declared schedule names n arcs and the day returned a different number of cells
`lsqvce:makeLoader:emptyDay`	without a schedule, a calendar day returned no arc in any stream

identifier	meaning
`lsqvce:makeLoader:badArc`	an arc id outside the declared schedule, or an arc of a multi-axis day that does not hold whole epochs
`lsqvce:makeLoader:badDayCep`	`Cep` is malformed or does not match the arc's rows
`lsqvce:makeLoader:axisConflict`	`nAxis` disagrees with `Cep` or does not divide the row count
`lsqvce:makeLoader:cepTwice`	the covariance spec carries a `Cep` and the day returns one
`lsqvce:covModel:cepOnVar`	a `var` law with a `Cep`
`lsqvce:makeLoader:noCep`	*(warning, once per run)* a multi-axis day without `Cep`

Errors in the declaration itself are raised by `fit` before any data is read and carry `lsqvce:fit:` identifiers (`unknownField`, `noSystems`, `noCommon`, `missingField`, `noDays`, `badId`, `duplicateSystem`, `builderKind`, `badArcs`, `badScope`, ...).

3 Covariance specifications

`data.systems(k).covariance.(obsType)` is one struct per observation type. Its `type` selects the **temporal law** — how the errors of one stream correlate down the epochs — and `sigma` its **level**. The per-epoch accuracy is not declared here: it travels with the day as `Cep` (chapter `buildDay`). The toolbox composes the covariance of an arc as C = D (R_t ⊗ I_d) D. Full reference: `doc lsqvce.engine.covModel`.

3.1 Laws

`type`	fields	describes	whitening cost (b rows per batch, m per arc)
`'diag'`	`sigma` (scalar, or a vector of per-row standard deviations)	white noise	O(b)
`'arma'`	`ar` = `[1 a₁ … a_p]`, `ma` (default 1), `sigma` (innovation std)	a known whitening filter	O(b·p)
`'psd'`	`f` (n_f×1, Hz, one-sided), `S` (n_f×1, unit²/Hz)	a measured noise spectrum	O(m³) per arc (Toeplitz) or O(b·p) (AR filter)
`'acf'`	`acf = @(dt, opt)`, `sigma`	correlation as a function of the lag	O(m³) per arc
`'var'`	`ar` (d×d×p), `sigma` (d×d)	vector AR: cross-axis temporal correlation (no `Cep`)	O(b·p·d²)
`'custom'`	`fcn = @(t, opt)`	any dense covariance	O(m³) per arc

`sigma` is always a **standard deviation**, never a variance.

3.2 Policy fields

field	values	default	meaning under `Estimate = "noise"`
`estimate`	`"law"`, `"scale"`, `"none"`	`"law"`	`"law"`: spectral shape and level are estimated; `"scale"`: the declared shape is kept, only the level is estimated; `"none"`: the stream is used exactly as declared
`arcFactors`	logical	`Options.ArcFactors`	per-arc variance factors for this stream

A per-row `sigma` vector makes the spec a fixed structure: it can carry a level but not a spectral shape, so `estimate = "law"` on it is refused (`lsqvce:fit:lawOnFixedStructure`). A `Cep` does not trigger this — per-epoch accuracy together with an estimated temporal law is the supported combination.

`Options.Pool` decides which streams share one estimated spectral shape; every stream keeps its own level.

3.3 How a law is whitened

- `Options.Noise.Whitener` selects how every `psd` law whitens — the estimated law and every declared one: `"toeplitz"` (default, exact), `"ar"` or `"arma"` (a fitted filter). Check an AR fit with `lsqvce.engine.checkPSD` (the relative error of the reproduced autocovariance should be below 1e-3).
- Filter laws are applied in batches of up to 20 000 rows that overlap by the filter memory (for an MA part, until the impulse response of 1/B(z) has decayed to rounding), so the result does not depend on the batching. Dense laws (`acf`, `custom`, Toeplitz `psd`) are factorised over the whole arc (exact; $O(m^3)$, m^2 memory: 3.2 GB at 20 000 epochs). An arc longer than the spec's `maxDenseArc` (default 20 000 epochs) is refused (`lsqvce:covModel:denseArcTooLong`): give a `psd` law a filter (`Whitener = "ar"`), cut the arcs shorter, or raise `maxDenseArc` when the memory is there.
- An `arma` law whose MA polynomial has a root on or outside the unit circle is refused (`lsqvce:covModel:maNotInvertible`).
- `Whitener = "arma"` reduces the AR fit of the spectrum to an ARMA (Durbin's method, `lsqvce.kernel.ar2arma`). A reduction that comes back non-stationary or with a non-invertible MA part falls back to the AR fit, with the warning `lsqvce:ar2arma:unstable` or `lsqvce:ar2arma:nonInvertibleMA`.
- Under `estimate = "law"` the declared law contributes only its `sigma` (the starting level): the shape is estimated from a white start.

4 `lsqvce.Options`

`lsqvce.Options` is a typed value class: every knob has a type, a range and a default, checked at assignment (`MATLAB:validators:*`) or at construction (`lsqvce:Options:*`). It says *how to run*; everything about the problem, including regularisation, is declared on `data`. Three equivalent spellings:

```
opts = lsqvce.Options(Estimate="noise", MaxPasses=6);   R = lsqvce.fit(data, opts);
opts.Noise.Grid = 120;                                   % change one later (validated)
R = lsqvce.fit(data, Estimate="noise", MaxPasses=6);     % name-value pairs straight on fit
```

`disp(opts)` lists every knob, marking the defaults with *.

4.1 What to estimate

knob	values	default	meaning
`Estimate`	`"none"`, `"noise"`	`"none"`	`"none"`: one generalised least-squares solve under the declared laws. `"noise"`: the joint variance-component estimator — per pool group the spectral shape of the noise, per stream its level, per arc a factor (with `ArcFactors`), and the σ of every prior group with `estimate = "scale"`.
`Pool`	`"all"`, `"obsType"`, `"stream"`	`"all"`	which streams share one estimated spectral shape: all streams of the run, the streams of one observation type across systems, or every (system, type) stream on its own. Every stream keeps its own level.
`ArcFactors`	logical	`true`	one variance factor per arc on top of the stream's level; the factor weights the solve. `false`: one level per stream. Turn off when arcs are few or short.

knob	values	default	meaning
`Robust`	`"none"`, `"huber"`, `"igg3"`, `"bisquare"`	`"none"`	M-estimation. With `Estimate = "none"`: stand-alone IRLS on the law-whitened innovations with a MAD scale per pass (budget `RobustOpts`, factors in `R.history(end).factor`). With `Estimate = "noise"`: a robust weight per epoch inside the joint estimator, on the residuals descaled by the declared accuracy (budget `MaxPasses`, factors in `R.noise.robust.factor`). Prefer `"igg3"` or `"bisquare"` for gross blunders.

4.2 The loop

knob	values	default	meaning
`MaxPasses`	integer ≥ 1	10	pass budget of the variance-component loop and of Gauss–Newton (IRLS uses `RobustOpts.MaxIter`). Exhausting it returns the last pass with `R.stopReason = "maxPasses"` and one warning.
`Tolerance`	> 0	1e-2	stop rule of the variance-component loop: max $\lvert\alpha^2 - 1\rvert$ over spectral bins and arcs; for Gauss–Newton the relative step in units of the formal error
`Probes`	integer ≥ 1	100	Hutchinson probes for the per-lag redundancies of the joint estimator
`Seed`	`[]` or double	`[]`	seed for the probes, recorded in `R.meta.seeds`

4.3 Run control

knob	values	default	meaning
`Backend`	`"qr"`, `"normal"`	`"qr"`	square-root information (Householder QR) or normal equations (Cholesky); the same numbers to rounding, QR the safer on ill-conditioned common blocks
`CacheDays`	> 0	1	built days kept resident per system; `Inf` keeps the calendar and builds each day once per run (a built type-day is m × nc doubles)
`Verbose`	logical	`false`	screen messages: 0 silent, 1 progress
`OnPass`	`[]` or `@(ctx)`	`[]`	inspection hook called after every pass of every iterative run (below)

4.3.1 `OnPass`

The hook receives one struct: `driver`, `pass`, `x`, `Cxx`, `sigma0`, `s0sq`; the loop state (noise loop: `C`, `Cgroups`, `sigma2`, `arcKeys`, `streams`, `sigma`, `white`, `whitePval`, `maxDev`, `skips`; IRLS: `factor`, `keys`, `scale`, `dx`; Gauss–Newton: `vPv`, `dxRel`, `lambda`); `acc`, the live accumulator; `data`, the declaration as passed to `fit`; `processingOptions{k}`, system k's as declared. Return nothing to continue, `false` to stop after the pass (`R.stopReason = "userStop"`). An error in the hook arrives as `lsqvce:callUser:threw` with your error as its cause.

4.4 The groups

A group is a struct; assign a whole or partial struct, or one field. A partial struct is merged onto the defaults; an unknown field is refused (`lsqvce:Options:badGroupField`).

4.4.1 `Noise` — the estimated stochastic model

field	default	meaning
`Grid`	60	spectral bins of the estimated law between 0 and Nyquist (its resolution), an integer ≥ 2; the law is exact at every lag of every arc whatever the Grid

field	default	meaning
`Whitener`	`"toeplitz"`	how every `psd` law (the estimated one and the declared ones) whitens an arc: exact Toeplitz Cholesky over the whole arc, or a fitted AR / ARMA filter (`"ar"`, `"arma"`)
`Smooth`	0	moving-average half-width on the estimated spectrum (0 = none)
`FreezeRedundancy`	1	a bin whose redundancy is below this keeps its value instead of being updated
`SkipThreshold`	0.2	the share of skipped arcs of a stream above which a pass is refused

4.4.2 `RobustOpts` — IRLS

field	default	meaning
`Tuning`	`[]`	tuning constant of the influence function; empty = Huber 1.345, IGG3 (1.5, 2.5), bisquare 4.685
`MaxIter`	30	IRLS passes
`Tol`	1e-8	relative change of x that ends the iteration

4.4.3 `Diagnostics` — the per-observation tests

field	default	meaning
`Alpha`	1e-3	significance level of the outlier test
`Bonferroni`	`true`	divide α by the number of tests
`Beta`	0.2	type-II error; the power of the minimal detectable bias is 1 − β
`RMin`	0.01	redundancy number below which an observation is untestable
`RWeak`	0.1	redundancy number below which it is weakly controlled
`Test`	`"auto"`	`"pope"` (σ_0 estimated), `"baarda"` (σ_0 known), `"mad"` (robust scale), or automatic
`RobustCheck`	`true`	run the MAD-scale check that flags a contaminated fit

field	default	meaning
`Store`	`"all"`	keep the per-observation arrays for all observations, only `"flagged"` ones, or `"none"`
`NWorst`	1000	how many worst observations `R.report` keeps

4.4.4 `Parallel`

field	default	meaning
`Mode`	`"serial"`	`"parfor"` or `"spmd"` with the Parallel Computing Toolbox; serial with one warning without it
`DistributeDays`	`false`	distribute the day loop of the accumulation over workers (bitwise-identical fold order)
`Threshold`	8	number of days below which distribution is not attempted

4.4.5 `GaussNewton` — problems with `data.x0`

field	default	meaning
`TolF`	1e-10	relative change of the objective that ends the iteration
`Lambda0`	1e-6	initial Marquardt damping (rows $\sqrt{\lambda \cdot \mathrm{diag}(J^T P J)} \cdot dx \approx 0$); a rejected step multiplies it by 10 (from 0: 1e-3), an accepted one divides it (below 1e-12: 0)

5 Priors

A prior is a group of pseudo-observations $L \cdot x = x_0 + v$ with weight $1/\sigma^2$, declared in your columns:

```
data.priors(j)            = struct(...)   % on the common parameters: L in the columns of A
data.systems(k).priors(l) = struct(...)   % on system k's local parameters: L in the columns of B
```

5.1 Fields

field	meaning
`group`	a regular expression on the name column of the table (`data.paramColumns.name`, default column 2). The toolbox builds L: one row per matched template row per instance (per day for a `'day'` row, per arc for an `'arc'` row, once for `'common'` / `'system'`).
`L`	the explicit operator (q × nc, or q × nl) in the columns `lsqvce.layout(data)` announces. Give `group` **or** `L`, never both.
`x0`	a-priori values (q × 1, or a scalar); default 0
`sigma`	ONE standard deviation for all rows of the group; relative weights inside a group go into L
`from`	the pass at which the group becomes active (default 1). A single solve reports a group with `from > 1` (`lsqvce:fit:priorNotActive`) and skips it.
`name`	label (default: the pattern); a local group becomes the stream `prior_<name>` of its system
`estimate`	`"none"` (default): σ fixed; `"scale"`: σ is estimated in every pass of `Estimate = "noise"`

Malformed fields, or an `L` of the wrong width, are refused by name (`lsqvce:fit:badPrior`).

5.2 Rules

- **Several rows make σ estimable.** A group with one row has redundancy ≤ 1; its σ is left unchanged and flagged in `R.noise.priorHistory`.
- **Local rows live in one day.** Every row of a local group has its non-zero columns inside one day's local block or the system block; days are coupled through common parameters only.
- **A datum is a group of its own, or several.** Minimum constraints $L = B^T$ on the null space, `x0 = 0`, give the minimum-constraint solution and covariance for any σ. The datum may be split over groups (translation and rotation declared separately): every group the data cannot see belongs to it, as long as it does not overlap a group the data does see. A group that only partly lies in the null space, or overlaps an information group, is information.
- **Datum rows are not observations.** Their redundancy is 0 by construction; the diagnostics keep them in Σr and vPv but do not test or count them (`D.nTests`, `D.datumRows`; quadrant `"datum"` in `R.reliability()`).

5.3 Where the result is reported

	common groups	local groups
σ per pass	`R.noise.priorHistory`	`R.noise.model{k}.sigma` of stream `satKey|prior_<name>`
final	`R.noise.priors`	the same stream entry

The rows of every prior enter `R.residuals()` and the redundancy count.

5.4 From a parameter table

A row of a GRAVFIRE-style parameter table carrying `{'Tikhonov', fromPass, sigma[, label]}` in its 11th column means exactly one such struct; `template/gravfire_data.m` emits them: rows sharing a label become one `group` with `estimate = "scale"`, rows without a label one fixed-σ group each.

6 `lsqvce.Results`

Every run of `lsqvce.fit` returns one `lsqvce.Results`. Its full help block, with every property and method, is in the chapter *Help of the user-tier units*; this chapter is the map.

6.1 Properties

property	size	content
`x`	nc×1	the common-parameter estimate
`sigma0`	scalar	a-posteriori standard deviation of unit weight
`stopReason`	string	`"single"`, `"converged"`, `"maxPasses"` or `"userStop"`
`converged`	logical	whether the iteration converged (true for a single solve)
`sol`	struct	the solve: `.x`, `.sigma0`, `.Cxx` (covariance scaled by σ_0^2), …
`diag`	struct	per-observation diagnostics: residuals in the whitened frame, redundancy numbers, test statistics, minimal detectable biases, the robust check
`history`	1×I struct	one record per pass of an iterative run (empty for a single solve)
`noise`	struct	the estimated stochastic model: per stream `.stream` keys and `.model{k}` (`.sigma` the level, `.C` the law's autocovariance); `.S` / `.Sgroups` the bin spectra per pool group; `.priors`, `.priorHistory`; `.robust` (influence, scale per stream, one factor per epoch)
`acc`	`BlockAccumulator`	the accumulator: re-solve, inspect blocks, residuals
`systems`	1×M struct	what ran per system: id, nl, day list, name
`meta`	struct	provenance: `.toolboxVersion`, `.matlabRelease`, `.createdUTC`, `.seeds`, `.options` (the resolved options), `.warnings`

6.2 Methods

method	returns
`R.report()`	the verdict and its evidence, levels 0 to 4 (`lsqvce.report`)
`R.summary()`	one row per stream: final σ, law, whiteness (iterative runs)
`R.convergence()`	one row per pass: the stream levels, whiteness and the stop-rule quantities `maxDev`, `levelDev` (iterative runs)
`R.coefficients(Alpha=a)`	estimates with formal errors and (1 − a) confidence bounds
`R.toTable(data)`	your parameter tables with estimate and standard deviation written in
`R.residuals(...)`	whitened (and raw) residuals per stream, re-read from the arcs
`R.contribution(...)`	the share of every group in the common solution, t_g = tr(N⁻¹N_g) (Level 2: per parameter)
`R.redundancy()`	the total redundancy n − u
`R.reliability()`	per observation: r, w, the minimal detectable bias and its quadrant
`R.systematic(...)`	unmodelled signal in the residuals: Ljung–Box, runs, binned means
`R.snoop(...)`	iterative data snooping with a clean re-solve; R is not changed
`R.compare(Rred)`	the nested-model F test against a reduced fit
`R.ellipse(i, j, ...)`	the confidence ellipse of a parameter pair
`R.plot(kind)`	`"residuals"`, `"contribution"`, `"psd"`, `"convergence"`: one figure per question

The readers that need an iteration (`summary`, `convergence`) raise `lsqvce:Results:noHistory` on a single solve; those that need the accumulator (`residuals`, `contribution`) raise `lsqvce:Results:noAccumulator` on the in-memory kinds.

7 Help of the user-tier units

The help blocks of the five user-tier units, verbatim: `help lsqvce.<name>` shows the same text and `doc lsqvce.<name>` the same page. Generated from the code at build time.

7.1 lsqvce.fit

LSQVCE.FIT Fit a least-squares adjustment (optionally with VCE) to declared data.

```
LSQVCE.FIT  Fit a least-squares adjustment (optionally with VCE) to declared data.

  R = lsqvce.fit(data)                        every default (one GLS solve)
  R = lsqvce.fit(data, opts)                  a lsqvce.Options object
  R = lsqvce.fit(data, Name, Value, ...)      the Options properties, straight on fit
  R = lsqvce.fit(data, opts, Name, Value)     both: the pairs override opts

  THE ONE VERB. You describe the problem with a plain struct `data`
  (the API reference), say what to do with lsqvce.Options, and
```

```
get a lsqvce.Results back:

    R = lsqvce.fit(data, Estimate="noise", Pool="obsType", OnPass=@myInspect);
    R.report;  R.plot("psd");  [C, L] = R.toTable(data);

THREE DATA KINDS, told apart by the fields of `data`, all on ONE accumulator:
  (A) streaming   data.commonParams, data.days, data.systems(k) with
                  buildDay / covariance / localParams / priors /
                  processingOptions and the optional arc schedule `arcs`
                  (buildDay returns one CELL per arc -- A, B, l, t,
                  Cep as 1 x nArc cells -- and the cells are the schedule
                  unless `arcs` declares one); optional data.priors,
                  data.paramColumns
  (B) in memory   data.A (m x u), data.l (m x 1), optional data.p (m x 1
                  weights); optional data.priors. A FULL m x m SPD weight
                  matrix is kind C's: return it from batchFcn
  (C) batched     data.batchFcn @(k) -> [A, l, p] (or @(k, x) -> [J, w, p] with
                  data.x0 for Gauss-Newton), data.numUnknowns, data.numBatches;
                  optional data.priors
WHAT RUNS (opts.Estimate / opts.Robust / data.x0):
  Estimate "none"    one generalised solve under the declared laws (adjustBlock)
  Estimate "noise"   the joint variance-component estimator (adjustVCEjoint):
                     one spectral shape per Pool group, one level per stream,
                     per-arc factors when ArcFactors; prior groups with
                     estimate "scale" get their sigma by Helmert / Koch-Kusche
  Robust "huber" | "igg3" | "bisquare"   IRLS on the same loop: alone it is
                     adjustRobust; with Estimate "noise" the robust pass runs
                     INSIDE the joint loop, R.noise.robust carries
                     the final factors
  data.x0            Gauss-Newton with damping (adjustNonlinear)
REGULARISATION is declared, never an option: data.priors(j) on the common
parameters and data.systems(k).priors(l) on the local ones -- one struct per
group, by `group` (a name pattern) or `L` (in the columns lsqvce.layout
announces), x0, sigma, from, name, estimate. A free network's datum is the
prior on its null space.
WATCHING A RUN: Options.OnPass fires after every pass with the pass state,
the live accumulator, `data` and every system's processingOptions.

INPUT
  data   1x1 struct    the declaration (kind A, B or C as above)
  opts   1x1 lsqvce.Options   default lsqvce.Options()
  Name, Value          any property of lsqvce.Options
OUTPUT
  R      1x1 lsqvce.Results   x, sigma0, sol (Cxx, dof, vtv), diag, acc, history,
                     noise (S, Sgroups, C, Cgroups, stream, model, priors), stopReason,
                              meta (options as resolved, warnings, seeds, version);
                              methods report, summary, convergence, residuals,
                              contribution, compare, plot, toTable, coefficients

ERRORS
  Your declaration is wrong (raised BEFORE any data is read):
    lsqvce:fit:unknownField    a field name the contract does not know
                               (names the field; typos are errors on
                               purpose)
    lsqvce:fit:noSystems       data.systems missing, not a struct, or empty
```

```
  lsqvce:fit:noCommon        data.commonParams missing, or not a 2-D
                             cell of template rows
  lsqvce:fit:badScope        a scope-token column 1 does not carry:
                             commonParams needs 'common'|'day' (raised
                             here), localParams needs
                             'arc'|'day'|'system' (raised by makeLoader
                             and relabelled)
  lsqvce:fit:badParamColumns data.paramColumns is malformed: not a
                             struct, an unknown field, only one of
                             start / end, or a column the table lacks
  lsqvce:fit:missingField    a required system field is absent, or
                             covariance carries no observation type
  lsqvce:fit:noDays          no day list anywhere
  lsqvce:fit:badId           id is not a numeric scalar or single char
  lsqvce:fit:duplicateSystem two systems share an id (explicit or
                             index default)
  lsqvce:fit:builderKind     buildDay is not a function handle
  lsqvce:fit:badArcs         arcs is a string (omit arcs: the cells are
                             the arcs), or a
                             resolver returned something that is not
                             unique positive integers
  lsqvce:fit:<makeLoader id> a makeLoader construction error, relabelled
                             (e.g. paramType, badArcs, localPlan);
                             the original is attached as the cause
Your choice is not available here:
  lsqvce:fit:notYet          Robust ~= "none" on a NONLINEAR batch problem
                             (x0): IRLS inside Gauss-Newton is not written;
                             linearise first
  lsqvce:fit:modelKind       Estimate = "noise" on kinds B / C, which have
                             no streams to estimate a law for
  lsqvce:batchSystem:weightMatrix  a batch returned a weight argument that
                             is neither m x 1 nor an m x m SPD matrix. A
                             full SPD P IS accepted on kind C and whitened
                             by its Cholesky factor -- raised
                             by lsqvce.engine.batchSystem
  lsqvce:fit:priorNotActive  (warning) a prior with from > 1 in a
                             single-pass run is not applied (a common
                             or a local group, WP-128)
  lsqvce:fit:badPrior        data.priors is malformed (unknown
                             field, L not p x nc, sigma not a positive scalar,
                             x0 of the wrong length)
  lsqvce:fit:badEstimate     covariance.<type>.estimate is not
                             "none" | "scale" | "law"
  lsqvce:fit:lawOnFixedStructure  estimate "law" on a stream whose
                             sigma is per-row (a fixed structure: level only)
  lsqvce:fit:varUnderNoise   a 'var' law under Estimate = "noise": the
                             joint sweep has no vector-AR path
  lsqvce:fit:badMatrix       kind B shapes disagree (A m x u, l m x 1,
                             p m x 1)
  lsqvce:fit:missingField    kind C lacks batchFcn / numBatches, or the
                             linear form lacks numUnknowns
  lsqvce:fit:uCount          kind C carries BOTH x0 and numUnknowns --
                             u is stated once (numel(x0))
Your buildDay is wrong (raised WHILE READING, by the engine, ids
unchanged): lsqvce:makeLoader:badDay / dayShape / badArc / badArcId /
```

```
    buildDay / badDayCep / axisConflict / cepTwice (the day's Cep and
    axis count), lsqvce:covModel:cepOnVar, lsqvce:accum:arcBadNc / arcBadNl;
    lsqvce:makeLoader:noCep is a once-per-run warning (a multi-axis day
    without Cep is composed with D = I)

  See also LSQVCE.OPTIONS, LSQVCE.RESULTS, LSQVCE.LAYOUT, LSQVCE.ENGINE.DECLARATION,
       LSQVCE.ENGINE.MAKELOADER, LSQVCE.ENGINE.ADJUSTBLOCK, LSQVCE.ENGINE.ADJUSTVCEJOINT,
           LSQVCE.ENGINE.ADJUSTROBUST, LSQVCE.ENGINE.ADJUSTNONLINEAR
```

7.2 lsqvce.Options

LSQVCE.OPTIONS The one settings object for LSQVCE.FIT: 12 knobs and 5 groups.

```
LSQVCE.OPTIONS  The one settings object for LSQVCE.FIT: 12 knobs and 5 groups.

  opts = lsqvce.Options()                        every default
  opts = lsqvce.Options(Name, Value, ...)        override some
  opts.Name = Value                              change one later (validated)
  R = lsqvce.fit(data, Name, Value, ...)         the same names, straight on fit

  WHAT TO ESTIMATE, HOW TO RUN. Everything ABOUT A STREAM (its law, whether
  its law / level / arc factors are estimated) is declared on the stream
  (data.systems(k).covariance.<type>: type, sigma, estimate, arcFactors);
  this object holds only RUN policy: 12 knobs plus five typed groups.

  TOP LEVEL
    Estimate   "none" | "noise"   "none": one GLS solve under the declared
                                  laws. "noise": the joint (Ellmer) variance
                                  component estimator on every stream whose
                                  spec allows it (estimate ~= "none"), with
                                  the policy below
    Pool       "all" | "obsType" | "stream"   which streams SHARE one estimated
                                  spectral shape (every stream keeps its own
                                  level regardless); default "all"
    ArcFactors logical            per-arc variance factors on (Ellmer 6.5.4),
                                  default true; false = one level per stream.
                                  The factor is the arc's ABSOLUTE Helmert
                                  factor and weights the solve, so sigma0
                                  settles near 1. TO READ AN ARC'S STANDARD
                                  DEVIATION back, use all three terms:
                                    declared sigma * sqrt(factor) * R.sigma0
    Robust     "none" | "huber" | "igg3" | "bisquare"   IRLS on the same
                                  loop, every kind (details in RobustOpts).
                                  With Estimate "noise" the robust pass runs
                                  INSIDE the joint loop: one residual sweep
                                  per pass, the factors in every arc's
                                  whitener on the next pass; MaxPasses is the
                                  budget, RobustOpts.Tuning the constants
                                  (RobustOpts.MaxIter / Tol belong to the
                                  stand-alone IRLS only)
    MaxPasses  double  10         the pass budget of the variance-component
                                  loop and of Gauss-Newton. IRLS has its OWN
                                  budget, RobustOpts.MaxIter
    Tolerance  double  1e-2       stop when max|alpha2 - 1| over frequencies
                                  and arcs is below it
    Backend    "qr" | "normal"    square-root vs normal equations
```

```
  Probes     double  100        Hutchinson probes for the per-lag
                                redundancies; an integer >= 1 (Ellmer's
                                value is 100). The joint estimator has no
                                exact route, so 0 is refused
  Seed       [] | double        RNG seed for the probes (recorded)
  CacheDays  double  1          built days kept resident per system; Inf =
                                the whole calendar (one build per run)
  Verbose    logical | 0 | 1    screen messages: 0 silent, 1 progress. The
                                value is passed to every driver as a flag
                                (logical(Verbose)), so 2 behaves as 1
  OnPass     [] | @(ctx)        YOUR inspection hook, called at the end of
                                EVERY pass of every iterative run (the
                                variance-component loop, IRLS, Gauss-Newton)
                                with ONE ctx struct: .driver .pass .x .Cxx
                                .sigma0, the loop state (noise: .C .Cgroups
                                .sigma2 .arcKeys .streams .sigma .white
                                .whitePval .maxDev .skips; IRLS: .factor
                                .keys .scale .dx; Gauss-Newton: .vPv .dxRel
                                .lambda), .acc (the LIVE accumulator: call
                                lsqvce.engine.residuals(ctx.acc, ...) or any
                                engine function inside the hook), .data (your
                                declaration as passed to fit) and
                                .processingOptions (per system, as declared).
                                Plot, drawnow, save wherever you like; return
                                nothing to continue, false to stop after this
                                pass (stopReason "userStop"). A throwing hook
                                is NOT swallowed (lsqvce:callUser:threw with
                                your error as cause)

GROUPS (structs; set fields one at a time or give a whole struct)
  Noise        Grid 60             spectral bins of the estimated law, 0 ..
                                   Nyquist (an integer >= 2); exact at every
                                   lag for any Grid
               Whitener "toeplitz"|"ar"|"arma"  how a PSD law whitens:
                                                the estimated law and every
                                                declared 'psd' law
               Smooth 0                         moving average on the PSD
               FreezeRedundancy 1               components below it hold
               SkipThreshold 0.2                skip RATE that refuses the pass
  RobustOpts   Tuning [], MaxIter 30, Tol 1e-8
  Diagnostics  Alpha 1e-3, Bonferroni true, Beta 0.2, RMin 0.01, RWeak 0.1,
               Test "auto"|"pope"|"baarda"|"mad", RobustCheck true,
               Store "all"|"flagged"|"none", NWorst 1000
  Parallel     Mode "serial"|"parfor"|"spmd", DistributeDays false, Threshold 8
  GaussNewton  TolF 1e-10, Lambda0 1e-6     (x0 problems; MaxPasses / Tolerance
               are the budget and the step tolerance)

REGULARISATION is not an option: it is DECLARED on the data, as
pseudo-observations in the user's own columns -- data.priors(k) on the
common parameters (L in the columns of A) and data.systems(k).priors(l) on
the local ones (L in the columns of B), each with x0, sigma, from, group.

METHODS
  engineArgs(target)   the Name-Value list a driver takes (validated)
  toStruct()           every public property as a struct (provenance)
```

```
  disp()               grouped display, engine defaults marked *

ERRORS
  MATLAB:validators:*          a typed property receives a value outside
                               its type / set / range (at assignment)
  lsqvce:Options:badGrid       Noise.Grid is not an integer >= 2
  lsqvce:Options:badVerbose    Verbose is neither logical nor an integer 0..2
  lsqvce:Options:badOnPass     OnPass is neither [] nor a function handle
  lsqvce:Options:badGroupField a group struct carries an unknown field
  lsqvce:Options:badTarget     engineArgs target is unknown

EXAMPLE
  opts = lsqvce.Options(Estimate="noise", Pool="obsType");
  opts.Noise.Grid = 2160;  opts.Parallel.Mode = "parfor";
  R = lsqvce.fit(data, opts);
  % Robust ~= "none" together with Estimate = "noise" runs the robust pass
  % inside the joint loop:
  opts.Robust = "huber";

See also LSQVCE.FIT, LSQVCE.RESULTS, the API reference, RELEASE_NOTES.md
```

7.3 lsqvce.Results

LSQVCE.RESULTS The outcome of a recovery: solution, diagnostics, provenance.

```
LSQVCE.RESULTS  The outcome of a recovery: solution, diagnostics, provenance.
  R = lsqvce.Results(sol, D, acc, diagSol, batchFcn, nBatch, systems)
  R = lsqvce.Results(sol, D, acc, diagSol, batchFcn, nBatch, systems, meta)

  Returned by every driver LSQVCE.FIT dispatches to -- LSQVCE.ENGINE.ADJUSTBLOCK,
  ADJUSTVCEJOINT, ADJUSTROBUST, ADJUSTNONLINEAR. It gathers
  everything a user needs to read and judge an adjustment in one object, and
  keeps the lower layers reachable for experts. You rarely construct it
  yourself; you receive it from adjustBlock and call R.report().

  PROPERTIES (read-only)
    x          nc x 1   double   Common-parameter estimate (the field).
    sigma0     1x1      double   A-posteriori variance factor (global).
    stopReason 1x1      string   Why the loop stopped: "single" |
                                 "converged" | "maxPasses". Convergence
                                 failure is a RESULT: the last pass is
                                 returned with its covariance and one warning.
    converged  1x1      logical  Did the pipeline converge? (true for a single
                                 solve; set by the iterative engines.)
   diag     1x1    struct  Per-observation diagnostics (LSQVCE.ENGINE.VERDICT.DIAGNOSE):
                                 redundancy, leverage, w-tests, MDB, ...
    history    1xI      struct   Per-iteration record (empty for a single
                                 solve; filled by LSQVCE.ENGINE.ADJUSTVCEJOINT:
                                 .maxDev .S .streams .sigma .white
                                 .whitePval per pass, .skips, .nIter, and
                                 under Robust the per-pass .robustScale (one
                                 column per stream) .nDown .nZero;
                                 by LSQVCE.ENGINE.ADJUSTROBUST: .scale .dx
                                 .nDown .nZero and, on the last pass, .factor
                                 .keys).
    noise      1x1      struct   Fitted noise models per stream (empty until
```

```
                              the variance-component loop populates it):
                              .stream (keys) .model{k} (.sigma the level,
                              .C the law's ACV) .S (the law: the bin
                              spectrum, LSQVCE.KERNEL.HATACV) .Sgroups /
                              .Cgroups (one per pool group)
                              .priors .priorHistory, and .robust
                              (.influence, .scale per stream,
                              .streams, .factor ONE PER EPOCH in sweep
                              order, .keys; empty fields under "none").
  sol        1x1      struct   Raw output of acc.solve() (.x, .sigma0, ...).
  acc        1x1      lsqvce.engine.BlockAccumulator   The accumulator (drop here
                              to re-solve, inspect blocks, get residuals).
  source     1x1      struct   Bridged solution from LSQVCE.ENGINE.VERDICT.SOURCE.
 batchFcn  1x1    function_handle  Re-reader used by LSQVCE.ENGINE.VERDICT.DIAGNOSE.
  nBatch     1x1      double   Batch count from the bridge.
  systems    1xM      struct   The resolved system specs (sysID/nl/dayList).
  meta       1x1      struct   Run provenance. Field list is
                              ADD-ONLY across versions, never renamed:
                              .toolboxVersion 1xK char  from Contents.m,
                                              the ONE version source
                              .matlabRelease  1xK char  version('-release')
                              .createdUTC     1xK char  ISO 8601, trailing Z
                              .seeds          1x1 struct every RNG seed the
                                              run resolved (struct() when
                                              none; a resolved-but-unset
                                              seed is stored as [])
                              .options        1x1 struct the RESOLVED
                                              options the run actually used
                              A bare construction (7 arguments) gets a
                              meta with empty seeds/options -- the field
                              is never missing.

METHODS
  V = R.report(Name,Value,...)   Print/return the level-0..4 verdict
                                 (LSQVCE.REPORT); options pass straight through.
  r = R.redundancy()             Total redundancy sum(r_i) = n - u.
  res = R.residuals(Name,Value)  Whitened/raw residuals (acc.residuals).
  con = R.contribution(...)      Per-group information share.
  T = R.convergence(Name,Value)  Per-ITERATION trajectory: when the loop
                                 settled (needs .history).
  F = R.compare(Rred)            Nested-model F test against a reduced fit.
  S = R.systematic(Name,Value)   Unmodelled signal in the residuals: Ljung-Box,
                                 runs, binned means.
  [D, REJ, sol] = R.snoop(...)   Iterative data snooping; R is not changed.
  T = R.reliability()            Per-observation r, w, MDB and (w, r) quadrant.
  E = R.ellipse(i, j, ...)       Confidence ellipse of the parameter pair (i, j).
  fh = R.plot(kind)             "residuals" | "contribution" | "psd" | "convergence"
                                 (one figure per question, nothing exported).
  [C, L, map] = R.toTable(data)  The expanded parameter tables with estimate and
                                 standard deviation written in.
  T = R.summary()                Per-STREAM final state: sigma, AR order,
                                 whiteness (needs .history).
  T = R.coefficients(Alpha=a)    The estimates as a table with formal
                                 errors and (1-a) confidence bounds
                                 (default a = 0.05).
```

```
  disp(R)                         One-screen summary.

THE TWO ITERATION READERS ARE COMPLEMENTARY. convergence() reduces across
STREAMS (one row per pass); summary() reduces across PASSES (one row per
stream). A converged loop can still hide a single stream that never
whitened -- the per-pass view reports the worst p-value without naming it.

EXAMPLE
  R = lsqvce.engine.adjustBlock(systems, days);
  R.report();
  if ~R.converged, lsqvce.util.log.warn('did not converge'); end
  x = R.x;   s0 = R.sigma0;

ERRORS
  lsqvce:Results:noVPv          compare() on a Results without a finite v'Pv
  lsqvce:Results:noHistory      an iteration reader was called on a
                                single-solve Results (adjustBlock does not
                                iterate); use the result of an iterating
                                fit (Estimate = "noise", Robust, Nonlinear)
  lsqvce:Results:noCovariance   coefficients() on a Results whose sol
                                carries no Cxx
  lsqvce:Results:noAccumulator  residuals()/contribution() on a Results
                                without an accumulator (fit kinds B / C)
  lsqvce:Results:noPerObservation reliability() on a fit whose diagnostics
                                kept no per-observation vectors

ACCURACY
  Results adds no numerics: x and sigma0 are the engine's (sigma0 is the
  A-POSTERIORI std of unit weight; sol.Cxx the covariance scaled by it --
  the streaming contract every producer must establish),
  report() is lsqvce.report on .diag, coefficients() is arithmetic on x
  and diag(Cxx) with z(0.05) = 1.959963984540054 in closed form.

See also LSQVCE.FIT, LSQVCE.ENGINE.ADJUSTBLOCK, LSQVCE.ENGINE.ADJUSTVCEJOINT, LSQVCE.REPORT,
         LSQVCE.ENGINE.VERDICT, LSQVCE.ENGINE.BLOCKACCUMULATOR
```

7.4 lsqvce.report

LSQVCE.REPORT Judge a least-squares adjustment and say so in words.

```
LSQVCE.REPORT  Judge a least-squares adjustment and say so in words.

  V = LSQVCE.REPORT(D) takes the struct from lsqvce.engine.verdict.diagnose and prints a
  verdict. It returns a struct of machine-readable findings so the same logic
  can drive an automated pipeline.

  The report is ordered so that a failure at one level invalidates the levels
  below it. It refuses to report parameter precision for an adjustment whose
  integrity checks failed, because a sigma computed from a broken system is
  not a conservative estimate -- it is a fabrication.

  LEVELS
    0  INTEGRITY   sum(r) == n-u, vPv >= 0, vPv consistent between passes.
                  These test the CODE, not the data. Failure here means
                  every number below is meaningless.
    1  GLOBAL      variance factor vs chi-square.
```

```
  2  LOCALISE    outliers (data snooping), with Bonferroni.
  3  RELIABILITY redundancy numbers, MDB, external reliability.
                 This is the level that says which observations you are
                 NOT ABLE TO CHECK.
  4  PARAMETERS  correlations, conditioning, precision.

NAME-VALUE
  'verbose'  true  print the report (default true)
  'width'    78    line width
  'Provenance' struct()  a run-provenance struct (LSQVCE.RESULTS .meta);
                    when non-empty, ONE level-0 NOTE line
                    (code 'PROVENANCE') states toolbox version, MATLAB
                    release, and creation time. R.report() forwards it
                    automatically; a bare lsqvce.report(D) prints none.
  'Iteration'  struct()  the loop state of an ITERATIVE run:
                    .converged, .stopReason, .nIter and optionally .what
                 (the loop's name; default 'stochastic model'). When non-empty and the
                    run was not a single solve, ONE level-1 line says
                    whether the stochastic model settled -- 'NOISE_SETTLED'
                    (NOTE) or 'NOISE_UNSETTLED' (WARN). Every number from
                    level 1 down rests on those weights, so a model that was
                    still moving qualifies all of them. R.report() forwards
                    it automatically. An optional .influence (char) names
                    the robust influence of an IRLS run; when non-empty ONE
                    level-1 NOTE ('ROBUST_FIT') says sigma0 and the
                    covariance are those of the reweighted problem.

OUTPUT V
  V.verdict   'PASS' | 'PASS WITH WARNINGS' | 'SUSPECT' | 'INVALID'
  V.findings  cell array of struct(level, severity, code, msg)
  V.nError, V.nWarn, V.nNote

INPUT
  D         1x1 struct  diagnostics from LSQVCE.ENGINE.VERDICT.DIAGNOSE
  varargin  Name/Value: 'verbose', 'width', 'alpha', 'rWeak', ...

EXAMPLE
    data = struct('batchFcn', src, 'numUnknowns', u, 'numBatches', 4);
    R = lsqvce.fit(data, Verbose=false);
    V = lsqvce.report(R.diag);         % prints the five levels (= R.report)

    % V.verdict    'PASS' | 'PASS WITH WARNINGS' | 'SUSPECT' | 'INVALID'
    % V.findings   machine-readable: {level, severity, code, msg}

    % Drive a pipeline off the codes rather than the text:
    codes = cellfun(@(f) f.code, V.findings, 'UniformOutput', false);
    if ismember('SNOOP_CONTAMINATED', codes)
        error('Pope is blind here -- re-run with test=mad.');
    end

  See also LSQVCE.ENGINE.VERDICT.DIAGNOSE, LSQVCE.ENGINE.VERDICT.SYSTEMATIC.
```

7.5 lsqvce.layout

LSQVCE.LAYOUT The column layout fit will use – so you can declare priors in it.

```
LSQVCE.LAYOUT  The column layout fit will use -- so you can declare priors in it.
  lay = lsqvce.layout(data)

  The toolbox derives from your parameter tables (scope tokens, calendar,
  arcs, validity) which column of A and of every system's B belongs to
  which instantiated parameter. This function announces that layout BEFORE
  fit runs, so that data.priors(j).L (columns of A) and
  data.systems(k).priors(l).L (columns of B) can be written in it -- the
  only knowledge the user's regularisation and the toolbox share.

  INPUT   data  1x1 struct  a kind-A declaration (the API reference)
  OUTPUT  lay   1x1 struct
    .nc            1x1        columns of A
   .common      nc x 1 struct (.row .day)    the template row and (for 'day' rows) the day of every col
    .systems(k)    1x1 struct per system:
       .nl         1x1        columns of B
     .local    nl x 1 struct (.row .day .arc) the template row / day / arc of every column of B
      .col      @(row, day, arc) -> column    lookup: the column of template row `row` on
                                         `day` (and `arc` for 'arc' rows; [] otherwise);
                                                  0 when that instance does not exist
    .colCommon    @(row, day) -> column        the same for A ([] day for 'common' rows)

  EXAMPLE  (a Tikhonov prior on local row 7 of system 1, on every day)
    lay = lsqvce.layout(data);
    nl  = lay.systems(1).nl;  rows = zeros(0, nl);
    for d = data.days
      c = lay.systems(1).col(7, d, []);  r = zeros(1, nl);  r(c) = 1;  rows(end+1, :) = r;
    end
   data.systems(1).priors = struct('L', rows, 'x0', 0, 'sigma', 0.01, 'name', 'xaccscale', 'estimate

  See also LSQVCE.FIT, LSQVCE.RESULTS/TOTABLE, the API reference
```

8 The engine, kernel and util tiers

What `fit` dispatches to and what it is built from: one line per public unit, from the `Contents.m` files (generated). `doc lsqvce.engine.<name>` opens the full page. Calls go downward only: the user tier calls the engine, the engine calls kernel and util.

8.1 lsqvce.engine — drivers, accumulator, loaders, sweeps, diagnostics

unit	what
`adjustNonlinear`	Gauss-Newton / Levenberg as a policy on the block path
`batchSystem`	a batch source as one local-free streaming system
`adjustRobust`	IRLS (Huber/IGG3/bisquare) as a policy on the block path
`adjustBlock`	one GLS solve over the streaming block-arrow pipeline
`adjustVCEjoint`	Ellmer ten-step joint co-estimation with keyed arc factors
`applyRegularisation`	regularisation of the reduced system
`arcVarianceFactors`	per-arc variance factors

unit	what
`BlockAccumulator`	block-arrow accumulation with per-system Schur pre-elimination
`BlockCofactor`	the cofactor of the full parameter set, kept as blocks
`checkPSD`	PSD sanity check
`collectResiduals`	residual collection per stream
`CommonPrior`	prior on the common block (regularisation as observations)
`contribution`	information contribution per group
`covComponent`	covariance component estimation
`covModel`	covariance spec -> whitening operators (per arc)
`datum`	datum defect detection and handling
`declaration`	validate a kind-A data struct and build its loaders
`gcvLambda`	GCV choice of the regularisation weight
`jointModelOpt`	joint-mode model -> loader UserOpt translation
`localDiagnostics`	diagnostics of the local blocks
`localColumns`	which (row, day, arc) owns each column of B
`makeLoader`	day-builder -> Loader (built inside fit from the data struct)
`mapreduce`	serial / parfor / spmd task runner with ordered reduce
`NormalEqSystem`	normal-equation backend
`outliers`	outlier listing from diagnostics
`psdVCE`	PSD amplitude + arc-factor VCE (Ellmer workhorse)
`QRSystem`	QR backend
`ReducedContribution`	reduced-normal contribution bookkeeping
`redundancy`	redundancy numbers (data pass)
`residuals`	second data pass: residuals and quadratic forms
`resolveNc`	common-parameter count derived from the systems
`synthLoader`	synthetic block-arrow loader (engine-tier fixture; leaves in M4)
`sharedProbe`	shared Hutchinson probe generator (RNG contract)
`streamContribution`	contribution per stream
`traceEstimate`	stochastic trace estimate
`vce`	Helmert group variance components (exact
`verdict`	the verdict: diagnostics, snooping, systematics, F test, ellipse
`runMeta`	immutable provenance record for Results.meta

8.2 lsqvce.kernel — the numerical kernels

unit	what
`ar2arma`	AR -> ARMA conversion
`armaACV`	ARMA autocovariance
`armaWhiten`	ARMA whitening filter (batched)

unit	what
`crossCorr`	cross-correlation of residual streams
`dctI`	DCT-I transform
`decorrFactor`	decorrelation factor of a dense covariance
`hatACV`	ACV of every bin of a piecewise-linear spectrum
`levinsonDurbin`	Levinson-Durbin recursion
`psd2acv`	PSD -> autocovariance
`redundancyLags`	redundancy per lag without the dense W^-1
`separableWhiten`	separable (temporal x axis) whitening
`stat`	distributions and test thresholds, base MATLAB only
`toeplitzCofactor`	Toeplitz cofactor construction
`varACV`	VAR autocovariance
`varWhiten`	VAR whitening
`whitenessCorr`	whiteness (Ljung-Box) statistic
`whittle`	multichannel Levinson (Whittle) VAR fit
`robustWeights`	IRLS weight factors from standardised residuals

8.3 lsqvce.util — plumbing

unit	what
`callUser`	call a user function: outputs counted once, errors wrapped
`getfielddef`	struct field with default
`toolboxVersion`	the version string from Contents.m (one source)
`onPass`	fire the per-pass inspection hook (Options.OnPass)
`rowWeightKey`	the one key convention for per-arc robust weights
`streamKey`	the one stream key 'satKey
`arcKey`	the one arc key of the joint estimator
`log`	toolbox-wide logging state
`mergeStruct`	struct merge (later wins)
`verbosity`	verbosity level resolution and formatting

9 Error and warning identifiers

Every identifier the package raises (generated from the identifier catalogue, which is kept closed both ways: nothing is raised that is not listed, nothing is listed that is not raised).

identifier	raised by	kind	introduced
`lsqvce:BlockCofactor:badBlocks`	BlockCofactor	error	v6.3.0
`lsqvce:BlockCofactor:multiSystemBlock`	BlockCofactor	error	v6.3.0
`lsqvce:Options:badGrid`	Options	error	v4.0
`lsqvce:Options:badGroupField`	Options	error	v4.0
`lsqvce:Options:badOnPass`	Options	error	v4.0
`lsqvce:Options:badTarget`	Options	error	v4.0
`lsqvce:Options:badVerbose`	Options	error	v4.0
`lsqvce:Results:badPlotKind`	Results	error	v4.0
`lsqvce:Results:noAccumulator`	Results	error	v4.0

identifier	raised by	kind	introduced
`lsqvce:Results:noCovariance`	Results	error	v4.0
`lsqvce:Results:noHistory`	Results	error	v4.0
`lsqvce:Results:noPerObservation`	Results	error	v6.5.0
`lsqvce:Results:noVPv`	Results	error	v4.0
`lsqvce:Results:tableMismatch`	Results	error	v4.0
`lsqvce:accum:arcBadNc`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:arcBadNl`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:arcNotFinite`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:arcRowMismatch`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badArc`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badCxx`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badLoader`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badMeta`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badPrior`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badSpecs`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badThresholds`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:badX`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:blockSchurFailed`	engine.BlockAccumulator	warning	v4.0
`lsqvce:accum:cancellation`	engine.BlockAccumulator	warning	v4.0
`lsqvce:accum:duplicateSystem`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:localNotRedundant`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:localSummarySkipped`	engine.BlockAccumulator	warning	v4.0
`lsqvce:accum:metaMismatch`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:noArcs`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:noDays`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:noStore`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:noSystems`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:notWholeEpochs`	engine.private.arcBatchWriter	Write	v4.0
`lsqvce:accum:pinvFallback`	engine.BlockAccumulator	warning	v4.0
`lsqvce:accum:priorNotPD`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:priorSize`	engine.BlockAccumulator	error	v4.0
`lsqvce:accum:unknownSystem`	engine.BlockAccumulator	error	v4.0
`lsqvce:adjustBlock:badCal`	engine.private.normalizeSystems	System	v4.0
`lsqvce:adjustBlock:badLoader`	engine.private.normalizeSystems	System	v4.0
`lsqvce:adjustBlock:badStruct`	engine.private.normalizeSystems	System	v4.0
`lsqvce:adjustBlock:badSystems`	engine.private.normalizeSystems	System	v4.0
`lsqvce:adjustBlock:noDays`	engine.private.normalizeSystems	System	v4.0
`lsqvce:adjustNonlinear:badX0`	engine.adjustNonlinear	error	v4.0
`lsqvce:adjustNonlinear:diverged`	engine.adjustNonlinear	warning	v4.0
`lsqvce:adjustNonlinear:maxIter`	engine.adjustNonlinear	warning	v4.0
`lsqvce:adjustNonlinear:nonFinite`	engine.adjustNonlinear	error	v4.0
`lsqvce:adjustRobust:collapsed`	engine.adjustRobust	error	v4.0
`lsqvce:adjustRobust:nonConverged`	engine.adjustRobust	warning	v4.0
`lsqvce:adjustVCEjoint:arcMismatch`	engine.adjustVCEjoint	error	v4.0
`lsqvce:adjustVCEjoint:badAxis`	engine.adjustVCEjoint	error	v4.0
`lsqvce:adjustVCEjoint:badS0`	engine.adjustVCEjoint	error	v6.7.0
`lsqvce:adjustVCEjoint:badSigma2`	engine.adjustVCEjoint	error	v4.0
`lsqvce:adjustVCEjoint:localNotPD`	engine.adjustVCEjoint	error	v4.0
`lsqvce:adjustVCEjoint:negCond`	engine.adjustVCEjoint	error	v4.0
`lsqvce:adjustVCEjoint:noArcs`	engine.adjustVCEjoint	error	v4.0
`lsqvce:adjustVCEjoint:noOverlap`	engine.adjustVCEjoint	error	v6.1
`lsqvce:adjustVCEjoint:nonConverged`	engine.adjustVCEjoint	warning	v4.0
`lsqvce:adjustVCEjoint:ridgeFactor`	engine.adjustVCEjoint	warning	v5.3
`lsqvce:adjustVCEjoint:notPD`	engine.adjustVCEjoint	error	v4.0

identifier	raised by	kind	introduced
`lsqvce:adjustVCEjoint:singularAdjustV`	engine.adjustVCEjoint	error	v4.0
`lsqvce:applyReg:badLambda`	engine.applyRegularisation	error	v4.0
`lsqvce:applyReg:badRegMatrix`	engine.applyRegularisation	error	v4.0
`lsqvce:applyReg:notPD`	engine.applyRegularisation	error	v4.0
`lsqvce:applyReg:sizeMismatch`	engine.applyRegularisation	error	v4.0
`lsqvce:applyReg:unknown`	engine.applyRegularisation	error	v4.0
`lsqvce:ar2arma:degenerate`	kernel.ar2arma	error	v4.0
`lsqvce:ar2arma:degenerateMA`	kernel.ar2arma	warning	v4.0
`lsqvce:ar2arma:nonInvertibleMA`	kernel.ar2arma	warning	v6.14.0
`lsqvce:ar2arma:tooShort`	kernel.ar2arma	error	v4.0
`lsqvce:ar2arma:unstable`	kernel.ar2arma	warning	v4.0
`lsqvce:arcVarianceFactors:arcMismatch`	engine.arcVarianceFactors	error	v4.0
`lsqvce:arcVarianceFactors:badMeanV`	engine.arcVarianceFactors	warning	v4.0
`lsqvce:arcVarianceFactors:clampV`	engine.arcVarianceFactors	warning	v4.0
`lsqvce:armaACV:nonStationary`	kernel.armaACV	error	v4.0
`lsqvce:armaACV:zeroLeadingAR`	kernel.armaACV	error	v4.0
`lsqvce:armaWhiten:acvTooShort`	kernel.armaWhiten	error	v4.0
`lsqvce:armaWhiten:badSigma`	kernel.armaWhiten	error	v4.0
`lsqvce:armaWhiten:sizeMismatch`	kernel.armaWhiten	error	v4.0
`lsqvce:armaWhiten:starkenNotPD`	kernel.armaWhiten	warning	v4.0
`lsqvce:batchSystem:badMatrix`	engine.batchSystem	error	v4.0
`lsqvce:batchSystem:badSingle`	engine.batchSystem	error	v4.0
`lsqvce:batchSystem:weightMismatch`	engine.batchSystem	error	v4.0
`lsqvce:callUser:badHandle`	util.callUser	error	v4.0
`lsqvce:callUser:threw`	util.callUser	error	v4.0
`lsqvce:callUser:tooFew`	util.callUser	error	v4.0
`lsqvce:checkPSD:badFit`	engine.checkPSD	warning	v4.0
`lsqvce:checkPSD:badModel`	engine.checkPSD	error	v4.0
`lsqvce:checkPSD:marginalFit`	engine.checkPSD	warning	v4.0
`lsqvce:checkPSD:unstable`	engine.checkPSD	warning	v4.0
`lsqvce:collectResiduals:badRes`	engine.collectResiduals	error	v4.0
`lsqvce:covComponent:badArg`	engine.covComponent	error	v4.0
`lsqvce:covComponent:lagTooLarge`	engine.covComponent	error	v4.0
`lsqvce:covModel:badCep`	engine.covModel	error	v4.0
`lsqvce:covModel:badPsdMode`	engine.covModel	error	v4.0
`lsqvce:covModel:badSigma`	engine.covModel	error	v4.0
`lsqvce:covModel:badWhitenMode`	engine.covModel	error	v4.0
`lsqvce:covModel:cepOnVar`	engine.covModel	error	v5.3
`lsqvce:covModel:denseAcfTooLong`	engine.covModel	error	v6.14.0
`lsqvce:covModel:degenerateFFT`	engine.covModel	error	v4.0
`lsqvce:covModel:maNotInvertible`	engine.covModel	error	v6.13.0
`lsqvce:covModel:negativePSD`	engine.covModel	warning	v4.0
`lsqvce:covModel:noACF`	engine.covModel	error	v4.0
`lsqvce:covModel:noAR`	engine.covModel	error	v4.0
`lsqvce:covModel:noCovForm`	engine.covModel	error	v4.0
`lsqvce:covModel:noFcn`	engine.covModel	error	v4.0
`lsqvce:covModel:noFilterForm`	engine.covModel	error	v4.0
`lsqvce:covModel:noPSD`	engine.covModel	error	v4.0
`lsqvce:covModel:noSigma`	engine.covModel	error	v4.0
`lsqvce:covModel:noType`	engine.covModel	error	v4.0
`lsqvce:covModel:noVAR`	engine.covModel	error	v4.0
`lsqvce:covModel:notWholeLags`	engine.covModel	error	v4.0
`lsqvce:covModel:psdSanitizeEscape`	engine.covModel	error	v4.0
`lsqvce:covModel:sigmaLength`	engine.covModel	error	v4.0

identifier	raised by	kind	introduced
`lsqvce:covModel:unknownType`	engine.covModel	error	v4.0
`lsqvce:crossCorr:shortSeries`	kernel.crossCorr	warning	v4.0
`lsqvce:datum:BnotNull`	engine.datum	warning	v4.0
`lsqvce:datum:autoDetected`	engine.datum	warning	v4.0
`lsqvce:datum:badB`	engine.datum	error	v4.0
`lsqvce:datum:rankDeficientB`	engine.datum	error	v4.0
`lsqvce:datum:zeroN`	engine.datum	error	v4.0
`lsqvce:dctI:tooShort`	kernel.dctI	error	v4.0
`lsqvce:decorrFactor:notPD`	kernel.decorrFactor	mixed	v4.0
`lsqvce:diagnose:badPmatrix`	engine.verdict	error	v4.0
`lsqvce:diagnose:badTest`	engine.verdict	error	v4.0
`lsqvce:diagnose:batchMismatch`	engine.verdict	error	v4.0
`lsqvce:diagnose:nBatch`	engine.verdict	error	v4.0
`lsqvce:diagnose:noRobustScale`	engine.verdict	error	v4.0
`lsqvce:diagnoseAccum:nonKeplerian`	engine.verdict	error	v4.0
`lsqvce:diagnoseAccum:noSystems`	engine.verdict	error	v4.0
`lsqvce:ellipse:badPair`	engine.verdict	error	v4.0
`lsqvce:ellipse:samePair`	engine.verdict	error	v4.0
`lsqvce:fit:badArcs`	engine.declaration	error	v4.0
`lsqvce:fit:badEstimate`	fit	error	v4.0
`lsqvce:fit:badId`	engine.declaration	error	v4.0
`lsqvce:fit:badMatrix`	fit	error	v4.0
`lsqvce:fit:badParamColumns`	engine.declaration	error	v4.0
`lsqvce:fit:badPrior`	fit	error	v4.0
`lsqvce:fit:badScope`	engine.declaration	error	v4.0
`lsqvce:fit:builderKind`	engine.declaration	error	v4.0
`lsqvce:fit:duplicateSystem`	engine.declaration	error	v4.0
`lsqvce:fit:lawOnFixedStructure`	fit	error	v4.0
`lsqvce:fit:missingField`	engine.declaration, fit	error	v4.0
`lsqvce:fit:modelKind`	fit	error	v4.0
`lsqvce:fit:noCommon`	engine.declaration	error	v4.0
`lsqvce:fit:noDays`	engine.declaration	error	v4.0
`lsqvce:fit:noSystems`	engine.declaration	error	v4.0
`lsqvce:fit:notYet`	fit	error	v4.0
`lsqvce:fit:priorNotActive`	fit	warning	v4.0
`lsqvce:fit:uCount`	fit	error	v4.0
`lsqvce:fit:varUnderNoise`	fit	error	v6.1.5
`lsqvce:fit:unknownField`	engine.declaration, fit	error	v4.0
`lsqvce:ftest:differentData`	engine.verdict	error	v4.0
`lsqvce:ftest:invalidPremise`	engine.verdict	warning	v4.0
`lsqvce:ftest:noRedundancy`	engine.verdict	error	v4.0
`lsqvce:ftest:notNested`	engine.verdict	error	v4.0
`lsqvce:ftest:zeroVariance`	engine.verdict	error	v4.0
`lsqvce:gcv:atLowerBound`	engine.gcvLambda	warning	v4.0
`lsqvce:gcv:atUpperBound`	engine.gcvLambda	warning	v4.0
`lsqvce:gcv:degenerate`	engine.gcvLambda	warning	v4.0
`lsqvce:gcvLambda:qzFallback`	engine.gcvLambda	warning	v4.0
`lsqvce:hatACV:badX`	kernel.hatACV	error	v6.7.0
`lsqvce:jointArcParts:sizeMismatch`	engine.private.jointArcParts	error	v4.0
`lsqvce:jointModelOpt:badS`	engine.jointModelOpt	error	v6.7.0
`lsqvce:jointModelOpt:badWidth`	engine.jointModelOpt	error	v4.0
`lsqvce:jointModelOpt:badStream`	engine.jointModelOpt	error	v4.0
`lsqvce:levinson:notPD`	kernel.levinsonDurbin	mixed	v4.0
`lsqvce:levinson:orderTooHigh`	kernel.levinsonDurbin	error	v4.0

identifier	raised by	kind	introduced
`lsqvce:localDiagnostics:notFinite`	`engine.localDiagnostics`	error	v4.0
`lsqvce:log:noParallelQueue`	`util.log`	warning	v4.0
`lsqvce:makeKeyFcn:badGroupBy`	`engine.private.makeKeyFcn`	error	v4.0
`lsqvce:makeLoader:arcCount`	`engine.makeLoader`	error	v6.0
`lsqvce:makeLoader:badArcs`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:badArcs`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:axisConflict`	`engine.makeLoader`	error	v5.3
`lsqvce:makeLoader:badDay`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:badDayCen`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:badPrior`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:badScope`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:badValidity`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:cepTwice`	`engine.makeLoader`	error	v5.3
`lsqvce:makeLoader:buildDay`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:dayShape`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:emptyDay`	`engine.makeLoader`	error	v6.0
`lsqvce:makeLoader:localPlan`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:ncConflict`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:nlConflict`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:noCep`	`engine.makeLoader`	warning	v5.3
`lsqvce:makeLoader:noArcs`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:noCov`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:noObsTypes`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:paramType`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:priorType`	`engine.makeLoader`	error	v4.0
`lsqvce:makeLoader:rowWeightSize`	`engine.makeLoader`	error	v4.0
`lsqvce:mapreduce:badTask`	`engine.mapreduce`	error	v4.0
`lsqvce:mapreduce:serialFallback`	`engine.mapreduce`	warning	v4.0
`lsqvce:normalEq:badMerge`	`engine.NormalEqSystem`	error	v4.0
`lsqvce:normalEq:badWidth`	`engine.NormalEqSystem`	error	v4.0
`lsqvce:normalEq:cancellation`	`engine.NormalEqSystem`	warning	v4.0
`lsqvce:normalEq:noData`	`engine.NormalEqSystem`	error	v4.0
`lsqvce:normalEq:noRedundancy`	`engine.NormalEqSystem`	error	v4.0
`lsqvce:normalEq:notSPD`	`engine.NormalEqSystem`	error	v4.0
`lsqvce:normalEq:sizeMismatch`	`engine.NormalEqSystem`	error	v4.0
`lsqvce:prior:badGroup`	`engine.CommonPrior`	error	v6.11.0
`lsqvce:prior:badTarget`	`engine.CommonPrior`	error	v4.0
`lsqvce:probeMatrix:badDist`	`engine.private.probeMatrix`	error	v4.0
`lsqvce:psd2acv:aboveNyquist`	`kernel.psd2acv`	warning	v4.0
`lsqvce:psd2acv:nonPositiveVal`	`kernel.psd2acv`	error	v4.0
`lsqvce:psd2acv:tooFewPoints`	`kernel.psd2acv`	error	v4.0
`lsqvce:psdVCE:arcMismatch`	`engine.psdVCE`	error	v4.0
`lsqvce:psdVCE:shapeMismatch`	`engine.psdVCE`	error	v4.0
`lsqvce:qr:badMerge`	`engine.QRSystem`	error	v4.0
`lsqvce:qr:badWidth`	`engine.QRSystem`	error	v4.0
`lsqvce:qr:cancellation`	`engine.QRSystem`	warning	v4.0
`lsqvce:qr:noData`	`engine.QRSystem`	error	v4.0
`lsqvce:qr:noRedundancy`	`engine.QRSystem`	error	v4.0
`lsqvce:qr:rankDeficient`	`engine.QRSystem`	error	v4.0
`lsqvce:qr:singularR`	`engine.QRSystem`	error	v4.0
`lsqvce:qr:sizeMismatch`	`engine.QRSystem`	error	v4.0
`lsqvce:reduced:notSquare`	`engine.ReducedContribution`	error	v4.0
`lsqvce:reduced:sizeMismatch`	`engine.ReducedContribution`	error	v4.0
`lsqvce:redundancy:hutchinson`	`engine.redundancy`	warning	v4.0

identifier	raised by	kind	introduced
`lsqvce:redundancy:hutchinsonExplodes`	`engine.redundancy`	warning	v4.0
`lsqvce:redundancy:identityCollinear`	`engine.redundancy`	warning	v4.0
`lsqvce:redundancy:noStore`	`engine.redundancy`	error	v4.0
`lsqvce:redundancy:smallGroup`	`engine.redundancy`	warning	v4.0
`lsqvce:redundancy:spmdNotConverged`	`engine.redundancy`	warning	v4.0
`lsqvce:redundancyLags:badEta`	`kernel.redundancyLags`	error	v4.0
`lsqvce:redundancyLags:badLevel`	`kernel.redundancyLags`	error	v4.0
`lsqvce:redundancyLags:badTol`	`kernel.redundancyLags`	error	v4.0
`lsqvce:redundancyLags:badW`	`kernel.redundancyLags`	error	v4.0
`lsqvce:residuals:badSol`	`engine.residuals`	error	v4.0
`lsqvce:residuals:noStore`	`engine.residuals`	error	v4.0
`lsqvce:residuals:sigmaMismatch`	`engine.residuals`	warning	v4.0
`lsqvce:residuals:supersededSol`	`engine.residuals`	warning	v4.0
`lsqvce:resolveNc:conflict`	`engine.resolveNc`	error	v4.0
`lsqvce:resolveNc:empty`	`engine.resolveNc`	error	v4.0
`lsqvce:resolveNc:missing`	`engine.resolveNc`	error	v4.0
`lsqvce:robust:atBreakdown`	`engine.private.warnAtBreakdown`	warning	v6.1.4
`lsqvce:robustWeights:badHuberC`	`kernel.robustWeights`	error	v4.0
`lsqvce:robustWeights:badTuning`	`kernel.robustWeights`	error	v4.0
`lsqvce:robustWeights:scaleFallback`	`kernel.robustWeights`	warning	v6.1.4
`lsqvce:runMeta:badArg`	`engine.runMeta`	error	v4.0
`lsqvce:separable:acvTooShort`	`kernel.separableWhiten`	error	v4.0
`lsqvce:separable:badSigma`	`kernel.separableWhiten`	error	v4.0
`lsqvce:separable:badSigmaShape`	`kernel.separableWhiten`	error	v4.0
`lsqvce:separable:epochNotPD`	`kernel.separableWhiten`	error	v4.0
`lsqvce:separable:notWhitened`	`kernel.separableWhiten`	error	v4.0
`lsqvce:separable:sigmaLength`	`kernel.separableWhiten`	error	v4.0
`lsqvce:separable:startNotPD`	`kernel.separableWhiten`	warning	v4.0
`lsqvce:sharedProbe:badProbe`	`engine.sharedProbe`	error	v4.0
`lsqvce:sharedProbe:notSquare`	`engine.sharedProbe`	error	v4.0
`lsqvce:snoop:breakdown`	`engine.verdict`	error	v4.0
`lsqvce:snoop:contaminated`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:correlatedBench`	`engine.verdict`	error	v4.0
`lsqvce:snoop:downdateFailed`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:forcedBreakdown`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:forcedContaminated`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:forcedOverlap`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:lowRedundancy`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:maxRejects`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:rejectFraction`	`engine.verdict`	warning	v4.0
`lsqvce:snoop:tooContaminated`	`engine.verdict`	error	v4.0
`lsqvce:stat:badDof`	`kernel.stat`	error	v4.0
`lsqvce:stat:badProb`	`kernel.stat`	error	v4.0
`lsqvce:stat:sizeMismatch`	`kernel.stat`	error	v4.0
`lsqvce:streamContribution:badScale`	`engine.streamContribution`	error	v4.0
`lsqvce:streamContribution:covNotPD`	`engine.streamContribution`	error	v4.0
`lsqvce:streamContribution:noAtrue`	`engine.streamContribution`	error	v4.0
`lsqvce:streamContribution:noStore`	`engine.streamContribution`	error	v4.0
`lsqvce:synthLoader:badArg`	`engine.synthLoader`	error	v4.0
`lsqvce:synthLoader:noNoiseModel`	`engine.synthLoader`	error	v4.0
`lsqvce:systematic:badCovariate`	`engine.verdict`	error	v4.0
`lsqvce:systematic:lbUnreliable`	`engine.verdict`	warning	v4.0
`lsqvce:systematic:noResiduals`	`engine.verdict`	error	v4.0
`lsqvce:systematic:tooFew`	`engine.verdict`	warning	v4.0

identifier	raised by	kind	introduced
`lsqvce:toeplitzCofactor:badVariance`	kernel.toeplitzCofactor	warning	v4.0
`lsqvce:toeplitzCofactor:empty`	kernel.toeplitzCofactor	error	v4.0
`lsqvce:toeplitzCofactor:notACV`	kernel.toeplitzCofactor	warning	v4.0
`lsqvce:traceEstimate:badOperator`	engine.traceEstimate	error	v4.0
`lsqvce:traceEstimate:negSize`	engine.traceEstimate	error	v4.0
`lsqvce:traceEstimate:nonSquare`	engine.traceEstimate	error	v4.0
`lsqvce:traceEstimate:sizeMismatch`	engine.traceEstimate	error	v4.0
`lsqvce:util:log:badKey`	util.log	error	v4.0
`lsqvce:varACV:badA`	kernel.varACV	error	v4.0
`lsqvce:varACV:notStationary`	kernel.varACV	error	v4.0
`lsqvce:varWhiten:badA`	kernel.varWhiten	error	v4.0
`lsqvce:varWhiten:badSigma`	kernel.varWhiten	error	v4.0
`lsqvce:varWhiten:badStartup`	kernel.varWhiten	error	v4.0
`lsqvce:varWhiten:notWhitened`	kernel.varWhiten	error	v4.0
`lsqvce:vce:badAccFcn`	engine.vce	error	v4.0
`lsqvce:vce:badSigma0`	engine.vce	error	v4.0
`lsqvce:vce:groupMismatch`	engine.vce	error	v4.0
`lsqvce:vce:hutchGroupMismatch`	engine.vce	error	v4.0
`lsqvce:vce:lowRedundancy`	engine.vce	warning	v4.0
`lsqvce:vce:notConverged`	engine.vce	warning	v4.0
`lsqvce:verbosity:badLevel`	util.verbosity	error	v4.0
`lsqvce:verbosity:badTag`	util.verbosity	error	v4.0
`lsqvce:whittle:badG`	kernel.whittle	error	v4.0
`lsqvce:whittle:notPD`	kernel.whittle	error	v4.0
`lsqvce:covModel:badNugget`	engine.covModel	error	v6.6.0
`lsqvce:covModel:nuggetNeedsWhiteMod`	engine.covModel	error	v6.6.0
`lsqvce:redundancyLags:badNugget`	kernel.redundancyLags	error	v6.6.0